

(Escriba sus datos en imprenta mayúscula)

Nombre y Apellido:

Legajo #: 25470/1

#Hoja 1/..... 4

Consideraciones: El parcial cuenta con 3 partes. Para aprobarlo se debe tener un puntaje de 6 o más en cada parte.

Parte A - Shell Scripting en GNU Linux 10

1. (4ptos) Realice un script en Bash que reciba como argumento una lista de nombres de usuario (se deberá validar que se reciba al menos uno), y para cada uno de los nombres de usuario recibidos que exista en el sistema, se deberá imprimir un reporte con la siguiente información:

- Nombre de usuario.
- Ruta al directorio personal (HOME), solo si el usuario tiene un directorio personal configurado y éste existe. Si no, informará el texto "Sin HOME". Pista: Recuerde el archivo visto que contiene esta información
- Cantidad de archivos (no directorios) con terminación ".docx" en su nombre que existen en su directorio personal y sus subdirectorios. Se deberá informar 0 si el usuario no posee un directorio personal o no existe.

2. (4ptos) Utilizando Bash, desarrolle las siguientes funciones para implementar una estructura de LISTA, la que deberá almacenarse en un arreglo:

- **init:** inicializa la lista. Puede recibir 0 o más parámetros que representan elementos iniciales para la lista.
- **push:** agrega 1 o más elementos al final de la lista.
- **last:** devuelve el último elemento de la lista, quitándolo de la misma.
- **first:** devuelve el primer elemento de la lista, quitándolo de la misma.
- **size:** devuelve el tamaño de la lista (cantidad de elementos en la misma).
- **print:** imprime todos los elementos de la lista.

Luego, escriba un cuerpo principal de script que cree una lista L sin elementos y utilice todas las funciones anteriores sobre L. No hace falta que presente un menú, simplemente ejemplifique cómo se invoca cada una de las funciones.

Recuerde validar en cada función que los parámetros y el estado de la lista sean adecuados para la operación que se intente realizar.

3. (2ptos) Escriba un script que reciba el path de un archivo por parámetro, debe validar que exista y, en caso afirmativo, informe qué permisos tiene el usuario sobre dicho archivo. En caso de tener permisos de ejecución, quitarlos.

Parte B - Memoria Virtual / E/S / File System 8.5

1. (3ptos) Dada la siguiente secuencia de referencias a páginas de un proceso, y contando con 4 frames de memoria, donde uno de ellos se reserva como frame de descarga asincrónica: 1, 3, 2, 1^M, 4, 2^M, 1^M, 5, 3, 4, 2, 4, 1^M

Realice el gráfico correspondiente al algoritmo de reemplazo LRU e indique la cantidad de fallos de página que se produjeron.

2. (3ptos) Suponga un disco rígido con 100 pistas numeradas de 0 a 99, donde la cabeza se encuentra en la pista 26 y viene de la 30. Se tiene el siguiente lote de requerimientos: { 28, 99, 11, 34, 9, 84, 31, 21, 23 } y luego de 40 movimientos más ingresa { 88, 2, 30 }. Indique la cantidad de movimientos de Seek para el algoritmo SCAN y realice el diagrama que permita justificar su respuesta.

3. Se tiene una unidad de disco con 4 platos, con 2 caras útiles cada uno, 12500 pistas por cara y 300 sectores por pista de 512 Bytes cada uno. Si el disco gira a 8000 RPM, tiene un tiempo de posicionamiento (seek) de 10,5 ms y una velocidad de transferencia de 146 MiB/seg (MebiBytes por segundo). Responder (se deben incluir los cálculos realizados para llegar a la respuesta):

- (0.5pto) Capacidad total del disco expresada en Bytes.
- (0.75pto) Cuántos milisegundos se tardarían en transferir un archivo almacenado de manera contigua de 512 sectores.
- (0.75pto) Cuántos milisegundos se tardarían en transferir un archivo almacenado de manera aleatoria de 320 sectores.

4. Si se tiene un disco con sectores de tamaño de 512 Bytes y un FileSystem que utiliza como unidad de asignación el cluster de 8 sectores. Suponga además que se tienen los archivos "Archivo1" que tiene un tamaño lógico de 28702 Bytes y a "Archivo2" con un tamaño lógico 45101 Bytes. Responda (se deben incluir los cálculos realizados para llegar a la respuesta):

- (1pto) ¿Cuántos clusters se necesitan para almacenar "Archivo1" y "Archivo2" en el disco?
- (1pto) ¿Cuánto habría de fragmentación interna al almacenar "Archivo1" y "Archivo2"? (indicarla en Bytes)

Parte C - Conceptos teóricos (1 pto. cada inciso) 6

1. En paginación por demanda, el Kernel es responsable de setear el valor del bit R en 1 cuando detecta el uso de una página.

2. En la técnica de Frecuencia de Fallo de Páginas, es condición suficiente que todos los procesos tengan una tasa de fallos superior a la cota superior definida para indicar que el sistema se encuentra en Hiperpaginación (Thrashing).

3. La ventaja en la memoria virtual que es implementada con paginación por demanda es que las páginas de un proceso pueden estar tanto en RAM (memoria principal) o en el área de swap para que el proceso pueda acceder a su contenido.

4. En el modelo del *working set* visto, la selección de una ventana Δ pequeña causará que (seleccione una opción) X no se llegue a tener la localidad que se mantengan páginas que no forman parte de la localidad.

5. La asignación de espacio indexada en un filesystem podrá causar fragmentación externa si se utilizan clusters de gran tamaño.

6. En la técnica de buffer caché de System V, un header permanecerá ligado a una única hash durante toda su vida.

7. En el diseño de la Entrada/Salida, el servicio de "Buffering" es brindado por el driver (controlador) de cada dispositivo.

8. Un FileSystem es un componente de software que implementa archivos teniendo en cuenta las diferencias entre cada medio de almacenamiento donde el FileSystem podrá usarse.

9. La técnica de reemplazo de páginas global resulta beneficiosa sobre la local, dado que permite aislar la tasa de fallo de páginas de cada proceso.

10. Según la implementación de File System de System V, ejecutar el comando "mv archi.doc nue.doc" causará que el i-nodo del directorio donde se encuentra el archivo archi.doc sufra cambios en algunos de sus atributos.

Parte a - 1

```
#!/bin/bash
```

```
if [ $# -lt 1 ];
```

```
Then
```

```
echo "Debes pasar al menos un parametro"  
exit 1
```

```
fi
```

```
for usu in $@;
```

```
do
```

```
cut -d: -f1 /etc/passwd | grep "$usu"
```

```
aux=$?
```

```
if [ $aux -eq 0 ]; # grep retorna 0 si encuentra una o mas coincidencias
```

```
Then
```

```
home=$(grep "$usu" /etc/passwd | cut -d: -f6)
```

```
if [ -d $home ];
```

```
Then
```

```
cat=$(find "$home" -name "*.*" -type f | wc -l)
```

```
echo "$usu, $home, $cat"
```

```
else
```

```
echo "$usu, Sin HOME, 0"
```

```
fi
```

```
fi
```

```
done
```

```
exit 0
```

↓
para agregar desc
errores

Parte A - 2

```
#!/bin/bash
```

```
init(){
```

```
lista=()
```

```
if [ $# -gt 0 ];
```

```
Then
```

```
lista+=("$@")
```

```
fi
```

```
return 0
```

```
}
```

```
push(){
```

```
if [ $# -eq 0 ];
```

```
Then
```

```
echo "Debes pasar al menos un parametro"
```

```
return 1
```

```
fi
```

```
lista+=("$@")
```

```
return 0
```

```
}
```



```

last(){
  cant = "${#lista[@]}"
  if [ $cant -eq 0 ];
  Then
    echo "la lista esta vacia"
    return 1
  Fi
  ult=$(( $cant - 1 ))
  elem = "${lista[$ult]}"
  unset lista[$ult]
  echo "$elem"
  return 0
}

```

```

First(){
  cant = "${#lista[@]}"
  if [ $cant -eq 0 ];
  Then
    echo "la lista esta vacia"
    return 1
  Fi
  elem = "${lista[0]}"
  unset lista[0]
  lista = (${lista[@]}) # reasigno indices
  echo "$elem"
  return 0
}

```

```

Size(){
  echo "${#lista[@]}"
  return 0
}

```

```

print(){
  if [ "${#lista[@]}" -eq 0 ]; Then echo "Lista vacia"
  else echo "${lista[@]}"
  Fi
}

```

```

init
last          # retorna 1
push          # retorna 1
push 1, 2, 3
size          # 3
print        # 1, 2, 3
last         # 3
first        # 1
size         # 1
print        # 2

```


Parte A - 3

```
#!/bin/bash
```

```
if [ $# -ne 1 ];
```

```
then
```

```
    echo "Debes pasar solo un parametro"
```

```
    exit 1
```

```
fi
```

```
path="$1"
```

```
if [ ! -f $path ];
```

```
Then
```

```
    echo "El path indicado no existe"
```

```
    exit 2
```

```
fi
```

```
if [ -r $path ];
```

```
Then
```

```
    echo "Lectura"
```

```
fi
```

```
if [ -w $path ];
```

```
Then
```

```
    echo "Escritura"
```

```
fi
```

```
if [ -x $path ];
```

```
Then
```

```
    echo "Ejecucion"
```

```
    chmod -x "$path"
```

```
fi
```


Parte B - 1

	M			M M			M						
	1	3	2	1	4	2	1	5	3	4	2	4	1
F1	1	1	1	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m
F2		3	3	3	4	4	4	5	5	5	2	2	2
F3			2	2	2	2 ^m	2 ^m	2 ^m	2 ^m	2 ^m	4	4	4
F4	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	1 ^m	3	3	3
PF	X	X	X		X			X	X	X	X		X

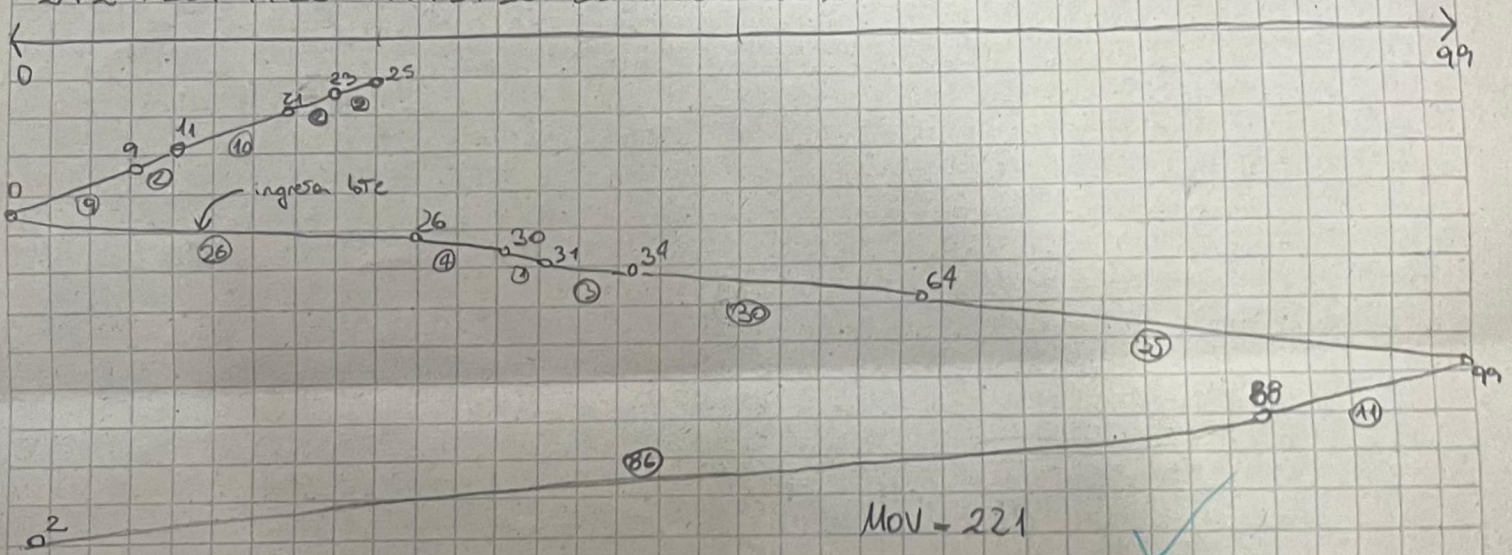
Cantidad de PF = 9



cola 1, 3, 2, 1, 4, 2, 1, 5, 3, 4, 2, 4, 1

Parte B - 2

$$2 + 2 + 10^m + 9 + 26 + 4 + 1 + 3 + 30 + 35 + 11 + 86 = 221$$



Parte B - 3

a. $4 \times 2 = 8$ cores

$$8 \times 12500 \times 300 \times 512 = 1536000000 \text{ bytes} = 1.536 \text{ TB}$$



S y C S/n

Parte B - 4

a.

$$1 \text{ cluster} = 8 \text{ sectors} \times 512 \text{ bytes} = 4096$$

$$\text{"Archivo 1"} = 7,007 = 8 \text{ clusters} \quad (28702 / 4096)$$

$$\text{"Archivo 2"} = 11,01 = 12 \text{ clusters} \quad (45101 / 4096)$$

Se necesitan 20 clusters para almacenar "Archivo 1" y "Archivo 2"

b.

$$(4096 \times 8) - 28702 = 4066 \text{ de Fi} \quad \text{"Archivo 1"}$$

$$(4096 \times 12) - 45101 = 4051 \text{ de Fi} \quad \text{"Archivo 2"}$$

genera un Total de 8117 bytes de fragmentación interna